Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

[**TODO:** Two to four sentences: the SSD and interface, QD=1 latencies, peak IOPS and throughput, where the queue-depth knee sits, and what tail latency and device state reveal.]

1 Introduction

[**TODO:** Motivation: the SSD under test, its interface ceiling, and why storage shows the same queueing-driven throughput–latency tradeoff as memory.]

This report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern and read/write mix shape IOPS, MiB/s and latency? (Sections 5 and 6)
3. Where is the queue-depth knee, and does Little's Law hold? (Section 7)
4. What do tail latency and device state (burst vs. steady state, temperature) reveal? (Sections 8 and 9)
5. Which setup mistake produces misleading numbers? (Section 10)

[**TODO:** One-paragraph summary of the main findings.]

2 Methodology and Safety

2.1 Platform

2.2 Data-loss safeguards

- Target: [**TODO:** dedicated empty partition or scratch directory on the SSD under test]. The run scripts refuse any other target (`scripts/safety.sh`): file targets must sit in a marked scratch directory, and raw devices must be named explicitly and be unmounted.
- [**TODO:** What else was done (nothing of value on the device, backups).]

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 2 threads/core (20 logical); NUMA: 1 node(s)
SMT	on (2 threads/core)
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: powersave; driver: intel_pstate; turbo: enabled; EPP: balance_performance
Pinning / isolation	[**TODO:** e.g. `taskset -c 2` on a P-core; isolcpus if used]; no isolcpus; runs pinned with taskset/–cpu
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.1
Tools	not available (not installed) (perf_event_paranoid 2); not available (not installed); MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	[**TODO:** model, capacity, firmware (`nvme id-ctrl` or `smartctl -i`)]
Interface	[**TODO:** NVMe PCIe generation and link width (`lspci -vv`, LnkSta), or SATA]
Target	[**TODO:** file on a filesystem (type, mount options) or raw partition; LBA range / file size]
Collected	2026-10-12 18:35 EDT

2.3 FIO configuration

Engine	[**TODO:** io_uring or libaio]; `direct=1` (page cache bypassed)
Alignment	random offsets aligned to the block size (`blockalign` defaults to `bs`)
Timing	?? s ramp (excluded) + ?? s measured; ?? repetitions per point
Data pattern	[**TODO:** incompressible (`refill_buffers`, 0% compressible) or other, held constant]
Latency metric	[**TODO:** completion latency (clat) or total latency]
Preconditioning	[**TODO:** sequential fill + sustained random writes; evidence in Section 9]
Job file	`SSD_Performance_Profiling/fio/job.fio` [1]

3 Validation and Sanity Checks

[**TODO:** Checks that the numbers describe the SSD, e.g.:]

- [**TODO:** Sequential throughput below the interface ceiling (PCIe generation × lanes, or SATA 6 Gb/s).]
- [**TODO:** FIO throughput agrees with `iostat -x` for one configuration.]
- [**TODO:** Effective queue depth (`iostat` aqu-sz) matches the requested iodepth × numjobs.]
- [**TODO:** Direct I/O confirmed (page cache does not grow during runs).]

Table 2: QD=1 performance (medians over repetitions; latencies in µs).

Data pending: `data/zero_queue.csv`
run the experiment in ACS-Code, then `scripts/sync_data.sh`

4 Zero-Queue Baselines

Hypothesis (written before measuring)

[**TODO:** Expected QD=1 latency for 4 KiB random and 128 KiB sequential reads and writes, and why reads and writes differ.]

Fixed	iodepth 1; numjobs 1; direct I/O
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

4.1 Analysis

[**TODO:** Latency composition at QD=1 (host stack, controller, flash read/program), variability, and caveats.]

5 Block-Size Sweep

Hypothesis (written before measuring)

[**TODO:** How IOPS, MiB/s and latency should change from 4 KiB to 1 MiB, for random vs. sequential access, and where the interface or controller limit should appear.]

Fixed	iodepth **??**; numjobs 1; direct I/O; reads [**TODO:** and writes]
Varied	block size 4 KiB–256 KiB (512 KiB, 1 MiB sequential); pattern
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`

5.1 Analysis

[**TODO:** Transition from IOPS-limited to throughput-limited behavior; the interface/controller ceiling; random vs. sequential gap and its causes.]

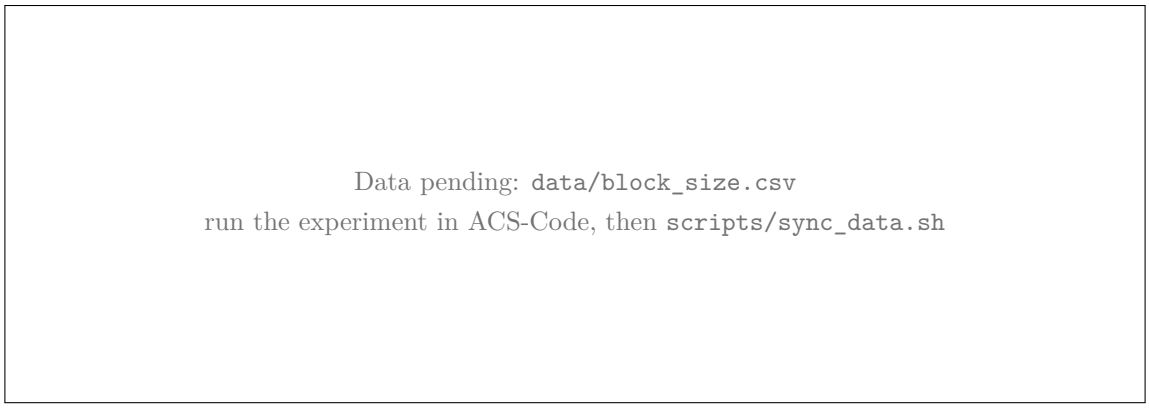


Figure 1: Block-size sweep. [**TODO:** takeaway; add the write side]

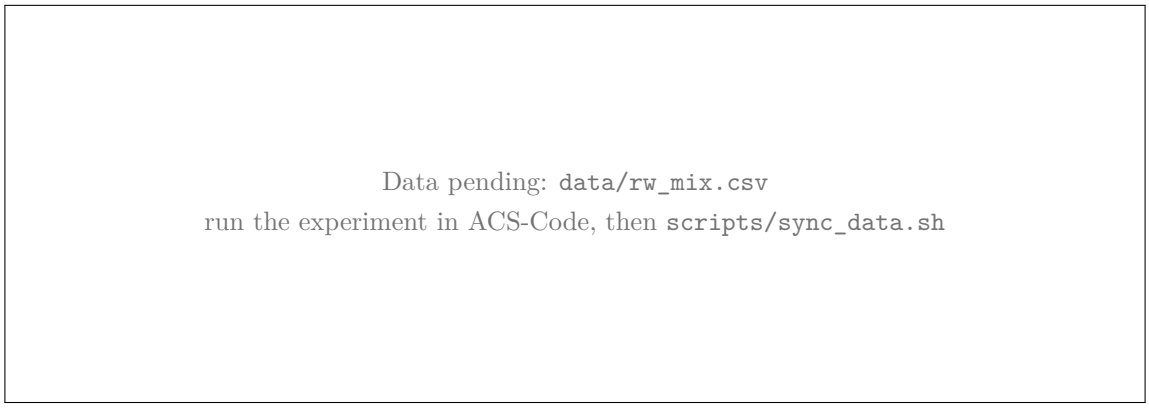


Figure 2: Read and write IOPS across mixes. [**TODO:** takeaway; add latency]

6 Read/Write Mix

Hypothesis (written before measuring)

[**TODO:** Expected IOPS and latency for 100% read, 70/30, 50/50 and 100% write at a fixed block size, and why mixing changes more than a weighted average would predict.]

Fixed	4 KiB random; iodepth **??**; numjobs 1; direct I/O
Varied	read percentage: 100, 70, 50, 0
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

6.1 Analysis

[**TODO:** Total IOPS and per-direction latency across mixes, and the controller/FTL behavior that explains them.]

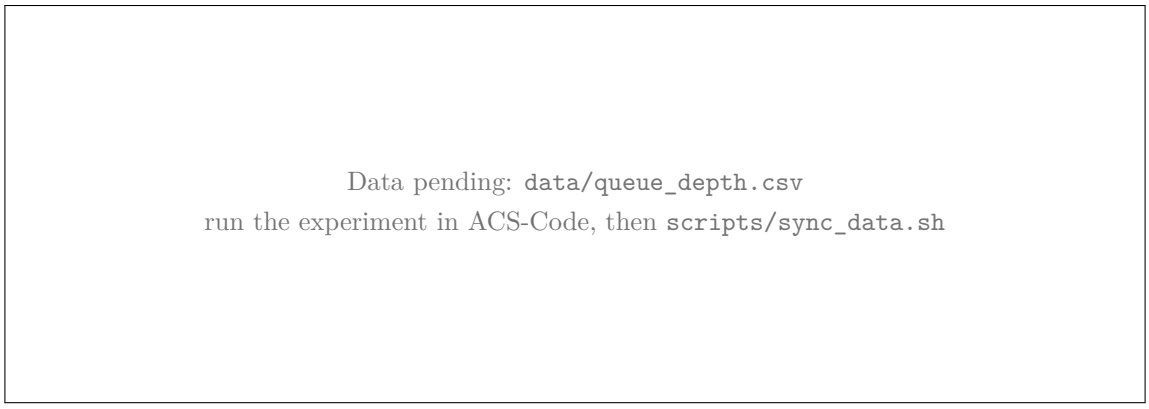
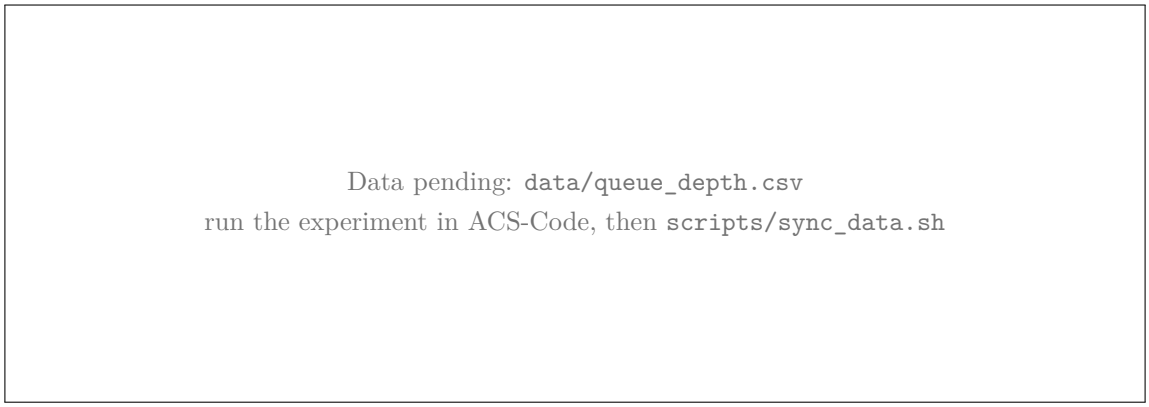


Figure 3: Throughput vs. latency; labels give the queue depth. [**TODO:** mark the knee]

Table 3: Little's Law: measured λW against the configured queue depth.



7 Queue-Depth Sweep

Hypothesis (written before measuring)

[**TODO:** Queue depth at which IOPS should saturate and latency rise sharply, and why (controller parallelism, NAND channels, host CPU).]

Fixed	4 KiB random read; numjobs 1; direct I/O
Varied	iodepth 1–256 [**TODO:** and/or numjobs]
Effective QD	[**TODO:** how verified (e.g. `iostat -x` aqu-sz)]
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`

7.1 Little's Law check

With the queue kept full, the number of outstanding I/Os L should equal iodepth $\times$ numjobs, so IOPS λ and mean latency W should satisfy $L = \lambda W$ [2].

7.2 Analysis

[**TODO:** Knee location and why; diminishing returns beyond it; where λW departs from the configured depth and what that reveals (host-side queueing, submission latency, engine limits).]

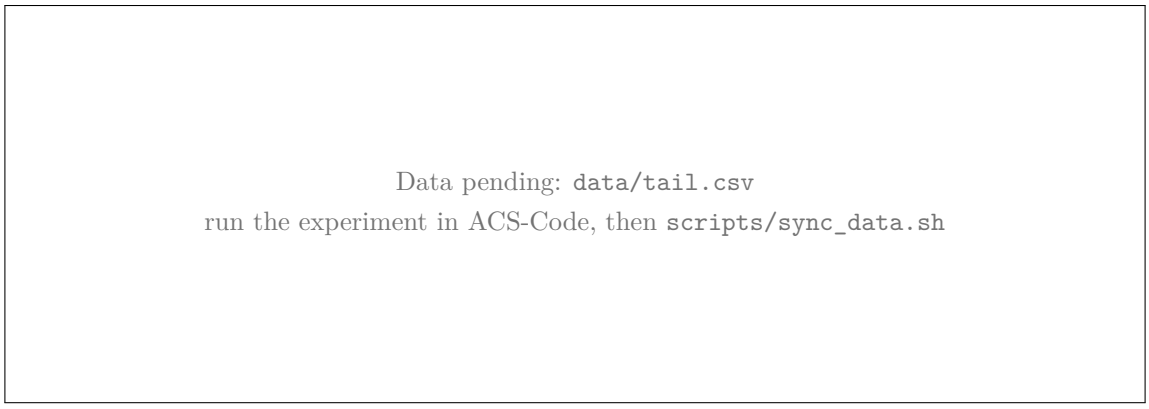


Figure 4: Latency percentiles at mid and near-knee queue depth. [**TODO:** takeaway]

8 Tail Latency

Hypothesis (written before measuring)

[**TODO:** How the gap between p50 and p99.9 should grow from a mid queue depth to the knee, and why.]

Fixed	4 KiB random read; numjobs 1; ?? s runs
Varied	iodepth: mid (??) and near-knee (??, from Section 7)
Script	`SSD_Performance_Profiling/scripts/run.sh tail`

8.1 Analysis

[**TODO:** Tail amplification with queueing; outliers from garbage collection or throttling; why tails matter more than means for this workload.]

9 Device State and Working Set

Hypothesis (written before measuring)

[**TODO:** When sustained writes should exhaust the SLC cache and how far throughput should drop; whether temperature should trigger throttling.]

Workload	4 KiB random write, iodepth 32, ?? s, per-second IOPS log
LBA range	?? GiB of ?? GiB capacity [**TODO:** effect of range / fill level]
State	[**TODO:** fresh vs. preconditioned; TRIM state]
Script	`SSD_Performance_Profiling/scripts/run.sh device_state` (and `precondition`)

9.1 Analysis

[**TODO:** Burst vs. steady-state throughput, SLC cache size estimate, garbage-collection or thermal effects, and what this means for the steady-state results in the earlier sections.]

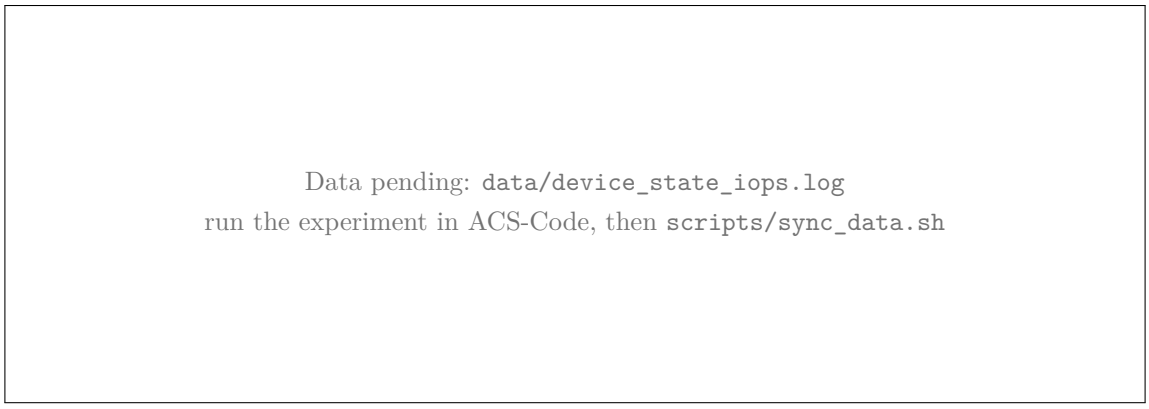


Figure 5: Sustained random-write IOPS over time. [**TODO:** mark SLC exhaustion / throttling]

Table 4: SMART snapshot before and after the sustained-write run.

Attribute	Before	After
Temperature (°C)	??	??
Data units written	??	??
Media errors	??	??

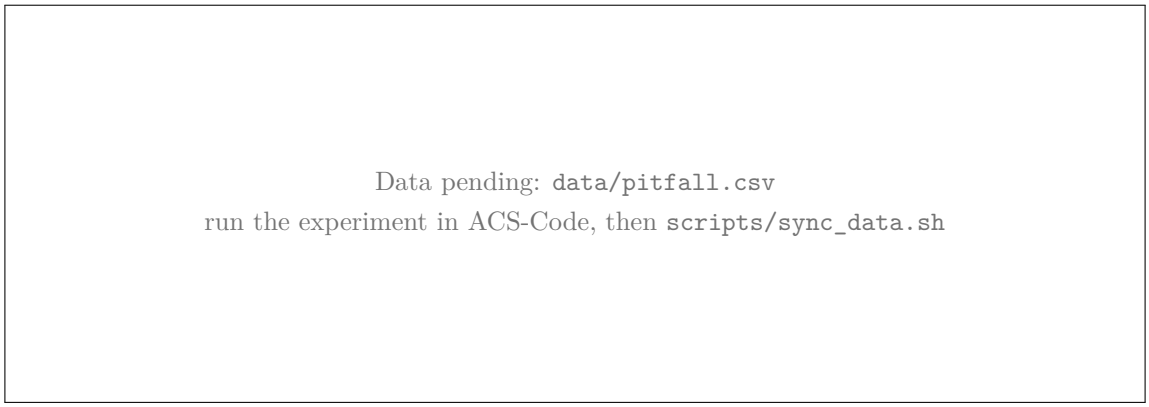
10 Misleading Setup

Hypothesis (written before measuring)

[**TODO:** What the flawed setup will report and why it looks plausible.]

Pitfall	[**TODO:** e.g. buffered I/O measuring the page cache, wrong effective queue depth, no preconditioning, thermal throttling]
Wrong setup	[**TODO:**]
Right setup	[**TODO:**]
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

Table 5: Direct vs. buffered I/O for the same workload.



10.1 Explanation

[**TODO:** Why the flawed numbers are wrong, how to detect the problem, and how the methodology avoids it.]

11 Anomalies and Limitations

11.1 Unexpected results

- [**TODO:** Observation → plausible hypothesis → how it was (or could be) tested.]

11.2 Limitations

- [**TODO:** e.g. one drive and firmware, filesystem overhead vs. raw device, limited LBA range, run length vs. steady state, host CPU as a bottleneck at high queue depth.]

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used
[**TODO:** Tools and models, and for which parts of the project.]

What they helped generate
[**TODO:** e.g. FIO job files, run and parsing scripts, debugging help.]

What was independently modified, tested, or rejected
[**TODO:** Concrete examples, including at least one AI suggestion that was wrong or misleading and how it was caught.]

Validation
[**TODO:** How measurements were checked independently (iostat cross-check, interface ceiling, effective queue depth).]

13 Conclusion

[**TODO:** Three to five sentences: what limits this SSD at low and high queue depth, how much tails and device state matter, and the most important lesson for benchmarking storage honestly.]

References

[1] J. Axboe, *Fio: Flexible I/O tester*, TODO: version used. [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, FIO job files, scripts and raw data: `github.com/SKempiak/ACS-Code`, commit [**TODO:** hash].

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
scripts/collect_env.sh
export TARGET=<scratch dir>/test.fio-scratch SIZE=<size> DEVICE=<nvme device>
SSD_Performance_Profiling/scripts/run.sh precondition
SSD_Performance_Profiling/scripts/run.sh all
scripts/sync_data.sh SSD_Performance_Profiling
```

Table 6: Raw data behind each figure.

Figure / table	Raw run directory (`SSD_Performance_Profiling/data/raw/...`)
Table 2	[**TODO:** `zero_queue/<timestamp>`]
Figure 1	[**TODO:** `block_size/<timestamp>`]
Figure 2	[**TODO:** `rw_mix/<timestamp>`]
Figure 3, Table 3	[**TODO:** `queue_depth/<timestamp>`]
Figure 4	[**TODO:** `tail/<timestamp>`]
Figure 5, Table 4	[**TODO:** `device_state/<timestamp>, precondition/<timestamp>`]
Table 5	[**TODO:** `pitfall/<timestamp>`]